Degree-Bounded NFA Compilation for Neuromorphic Pattern Matching

https://github.com/sh7vashrestha/TOC-MidTerm

Shiva Shrestha
Department of Computer Science
Kennesaw State University
Marietta, GA, USA
sshres16@students.kennesaw.edu

***Abstract*—In high-speed network intrusion detection, the traffic is scanned with regular expressions. The traditional approach turns each of these complex regular expressions into a Nondeterministic Finite Automaton (NFA), and then into a Deterministic Finite Automaton (DFA), as a DFA needs only one lookup table per input character, with $O(1)$ processing time per character. The main problem is that the subset construction can turn an n-state NFA into a DFA with up to 2^n states. To bypass this DFA state explosion bottleneck, emerging non-von Neumann architectures, such as spatial Automata Processors and neuromorphic memristor crossbar arrays, allow for the native, parallel execution of NFAs directly in hardware. However, translating abstract mathematical models to physical chips introduces routing and wiring constraints. Standard conversion algorithms assume a theoretical state can have an unlimited number of incoming or outgoing edges. In order to solve this, I defined a k-bounded NFA (k-NFA), in which no state has more than k = 2 incoming or outgoing edges, and gave an algorithm that converts any NFA into a 2-NFA by adding small trees of ε-states. I proved that the language is unchanged and that the cost is linear: one extra state and edge for each edge above the limit. For $(a|b)c(a|b)(a|b)(a|b)$, the Thompson NFA already meets the bound (26 states, 29 transitions); a second example shows states being added. The major limitation of 2-NFA is the delay in the signals that needs to pass through all of those ε-states.**

***Index Terms*—DFA, NFA, Thompson NFA, k-NFA, automata processing, fan-in, fan-out**

I. Introduction

Deep packet inspection (DPI) is generally used in the high speed intrusion detection system (IDS) to checks network packets against patterns written as complex regular expressions. The classical approach turns each of these regular expressions into a NFA, and then these NFAs are converted to DFA. A DFA reads each character with one table lookup, but converting an n-state NFA to a DFA can produce up to 2^n states (Hopcroft, Motwani, & Ullman, 2007).

Running the NFA directly avoids this problem, however it is slow on a normal computer. In a von Neumann processor, the transitions of every active state are fetched over the same processor-memory bus, so the processor mostly waits for memory. Spatial hardware gives each state its own circuit, so all active states update at once (Dlugosch, Brown, Glendenning, Leventhal, & Noyes, 2014). The new problem is wiring: each hardware node supports only a few input and output connections.

Our question is: how can any NFA be turned into a k-NFA, with at most $k = 2$ incoming and 2 outgoing edges per state, without changing its language? I answer with ε-states that split and merge edges, a correctness proof, and exact cost counts. I treat $k = 2$ as the target limit; real devices may differ from our assumptions.

II. Background & Literature Review

In 1968, Thompson showed how to build an NFA from a regular expression piece by piece (Thompson, 1968). Early Deep packet inspection (DPI) work has also made DFAs smaller and faster, with Kumar, Turner, and Williams (2006) introducing content-addressed delayed-input DFAs to cut storage, Yu, Chen, Diao, Lakshman, and Katz (2006) rewriting and grouping patterns to limit state explosion, and Brodie, Taylor, and Cytron (2006) designing a pipelined, compressed state machine for Application-Specific Integrated Circuits (ASICs) and Field-Programmable Gate Arrays (FPGAs).

On the other hand, Dlugosch et al. (2014) built the Automata Processor, which ran NFAs directly through programmable routing. In that work, authors linked routing capacity to delay, chip size and power, and split states that have too many inputs. Connection limits are therefore a known problem. Thus, our contribution in this work is a precise degree-2 model with exact costs, not routing trees themselves.

There also have been some recent work that targets other bottlenecks. Huang, Chen, Li, and Yang (2022) designed CAMA to match states in content-addressable memory. Similarly, Kong et al. (2022) and Le Glaunec, Kong, and Mamouras (2023) used the counters and bit vectors for repeated patterns, whereas Zhang, Tang, and Peng (2024) processed several characters per step on FPGAs. None of them bounds every state's in- and out-degree, which is the main goal of this work.

At last, on the neuromorphic side, Xia and Yang (2019) reviewed memristive crossbars that compute inside memory, Schuman et al. (2017) surveyed neuromorphic hardware

and its software, while Neftci et al. (2013) maped state machines onto imperfect neural circuits. This suggests separating two questions: does the compiled automaton accept the right language, and is the circuit reliable? I answer the first question in this work.

III. Methods

A. The k-NFA

An NFA is defined as $N = (Q, \Sigma, E, s, F)$: having Q states, an alphabet Σ with $\varepsilon \notin \Sigma$, a start state s, accepting states $F \subseteq Q$, and transitions $E \subseteq Q \times (\Sigma \cup \{\varepsilon\}) \times Q$. I write the transition relation δ as this edge set, so $|\delta| = |E|$.

A k-NFA is on the other hand defined as the tuple $N_k = (Q_k, \Sigma, E_k, s_k, F_k, k)$ with one extra rule: every state q should satisfy the condition: $d^-(q) \le k$ and $d^+(q) \le k$, where d^- and d^+ count incoming and outgoing edges, respectively (ε-edges included; a self-loop counts once on each side). In this work, I use $k = 2$. It runs like any ε-NFA: start from the ε-closure of $\{s\}$, follow every a-edge on input a, take the ε-closure again, and accept if an accepting state is active at the end.

B. Conversion algorithm

Algorithm 1 describes how the conversion from NFA to a 2-NFA happens. Every original edge gets a unique ID, so that its two endpoints can be moved independently. Similarly, For each state, ATTACH keeps up to two edges directly on the state. If there are more than 2 edges, it splits them into two nearly equal groups and hangs each group of two or more on a new ε-state: a splitter for outgoing edges, a merger for incoming edges. It repeats until and unless each and every state holds at most two edges. At last, each original labeled edge is reconnected between its new endpoints. New states are never accepting, so s and F are unchanged.

Algorithm 1 Convert an NFA into a 2-NFA

Require: NFA $N = (Q, \Sigma, E, s, F)$
Ensure: 2-NFA $N_2 = (Q_2, \Sigma, E_2, s, F)$ with $L(N_2) = L(N)$

$Q_2 \leftarrow Q$, $E_2 \leftarrow \emptyset$
for all edges $e = (p, a, q) \in E$ **do**
  $src[e] \leftarrow p$, $dst[e] \leftarrow q$
end for
for all states $q \in Q$ **do**
  ATTACH(q, outgoing edges of q, out)
  ATTACH(q, incoming edges of q, in)
end for
for all edges $e = (p, a, q) \in E$ **do**
  add $(src[e], a, dst[e])$ to E_2
end for
check $d^-(v) \le 2$ and $d^+(v) \le 2$ for every $v \in Q_2$
return N_2

procedure ATTACH(r, D, dir)
  if $|D| \le 2$ **then**
    for all $e \in D$ **do**
      if $dir =$ out **then**
        $src[e] \leftarrow r$
      else
        $dst[e] \leftarrow r$
      end if
    end for
  else
    split D into two nearly equal halves D_1, D_2
    for all $D_i \in \{D_1, D_2\}$ **do**
      if $|D_i| = 1$ **then**
        ATTACH(r, D_i, dir)
      else
        create a new state x and add it to Q_2
        if $dir =$ out **then**
          add (r, ε, x) to E_2 ▷ splitter
        else
          add (x, ε, r) to E_2 ▷ merger
        end if
        ATTACH(x, D_i, dir)
      end if
    end for
  end if
end procedure

C. Proof that the language is unchanged

Degree bound: After ATTACH procedure is executed, an original state keeps at most two edges on each side. A split state has one incoming and two outgoing edges; a merge state has two incoming and one outgoing. So no state exceeds two, even when the self-loops are counted.

(1)$L(N) \subseteq L(N_2)$: Each original edge $p \xrightarrow{a} q$ becomes a path: ε-edges through p's split tree, the same a-edge, then ε-edges through q's merge tree. Since the added edges will read no additional input, thus an accepting run of N becomes an accepting run of N_2 on the same word.

(2)$L(N_2) \subseteq L(N)$: A split tree leads only to its own state's outgoing edges, and a merge tree leads only to its own state. So an accepting run of N_2 breaks into pieces between original states, each holding exactly one original edge. Replacing each piece by that edge gives an accepting run of N on the same word. New states never accept, so N_2 cannot accept early.

Since $L(N) \subseteq L(N_2)$ and $L(N_2) \subseteq L(N)$, hence $L(N_2) = L(N)$, and since every regular language has an NFA, every regular language should also have an equivalent 2-NFA.

D. Cost

Let $n = |Q|$, $m = |E|$ and $[x]_+ = \max(0, x)$. Now, Spreading d edges with a binary tree takes $d - 2$ new states and $d - 2$ new ε-edges, so the number of added states is given by:

$$A = \sum_{q \in Q} \Big([d^-(q) - 2]_+ + [d^+(q) - 2]_+ \Big), \qquad |Q_2| = n + A, |E_2| = m + A. \tag{1}$$

Since $A \le 2m$, the 2-NFA has at most $n + 2m$ states and

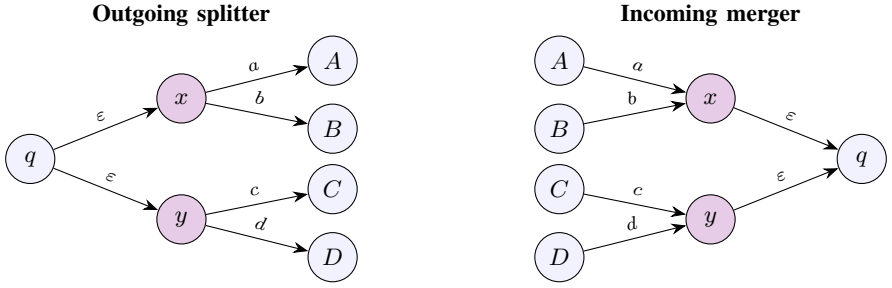


Fig. 1. Binary ε-routing for four edges. Purple helpers have degrees (1,2) for splitters or (2,1) for mergers.

$3m$ edges and is built in $O(n+m)$ time, versus up to 2^n DFA states. Then, each of the edges gains at most $\lceil \log_2 d \rceil - 1$ ε-steps per side. The helpers must remain ε-states, since removing ε-edges afterwards would bring back high degrees.

E. Implementation

I did the implementation of the pipeline in standard-library Python. It checks not only the degree bound but also exact language equivalence for every machine, and nine tests pass, including 30 random NFAs with cycles. I report graph sizes as mentioned in the requirement list of the assessment, not the chip speed or energy.

IV. Results & Analysis

A. Thompson NFA and 2-NFA

For $R = (a|b)c(a|b)(a|b)(a|b)$ over $\Sigma = \{a, b, c\}$, the language has the combination of $2^4 = 16$ words.

Now, the thompson's NFA construction generates 26 states (18 for the nine letters, 8 for the four unions) and 29 edges (9 symbol edges and 20 ε-edges). The start state is q_4 and the accepting state is q_{25} as illustrated in the figure 2. States are numbered in the order the construction creates them, and the start state of each union is created after its two branches, so the start state is q_4; the accepting state is q_{25}.

Every state already has at most 2 incoming transitions and 2 outgoing transitions, which means that $A = 0$, and the result will be identical to the original machine. Table I thus doubles as Table 1A (Thompson NFA) and Table 1B (2-NFA), demonstrating with d^- and d^+ values that states cannot have more than 2 transitions, as illustrated in Figure 2.

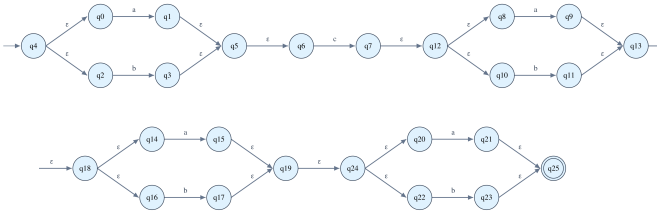


Fig. 2. The Thompson NFA for $(a|b)c(a|b)(a|b)(a|b)$, which is also its 2-NFA ($A = 0$), drawn in two rows (q_{13} continues to q_{18}). Every state has at most two incoming and two outgoing edges.

TABLE I
Table 1A (Thompson NFA) = Table 1B (2-NFA): transitions and degree audit. → marks the start state, ∗ the accepting state.

State	Symbol	ε	d^-	d^+
q_0	a:q_1	–	1	1
q_1	–	q_5	1	1
q_2	b:q_3	–	1	1
q_3	–	q_5	1	1
$\rightarrow q_4$	–	q_0, q_2	0	2
q_5	–	q_6	2	1
q_6	c:q_7	–	1	1
q_7	–	q_{12}	1	1
q_8	a:q_9	–	1	1
q_9	–	q_{13}	1	1
q_{10}	b:q_{11}	–	1	1
q_{11}	–	q_{13}	1	1
q_{12}	–	q_8, q_{10}	1	2
State	Symbol	ε	d^-	d^+
q_{13}	–	q_{18}	2	1
q_{14}	a:q_{15}	–	1	1
q_{15}	–	q_{19}	1	1
q_{16}	b:q_{17}	–	1	1
q_{17}	–	q_{19}	1	1
q_{18}	–	q_{14}, q_{16}	1	2
q_{19}	–	q_{24}	2	1
q_{20}	a:q_{21}	–	1	1
q_{21}	–	q_{25}	1	1
q_{22}	b:q_{23}	–	1	1
q_{23}	–	q_{25}	1	1
q_{24}	–	q_{20}, q_{22}	1	2
q_{25}^*	–	–	2	0

States were numbered in creation order; each union's start was built after its two branches in our python implementation, so the start is q_4, and not q_0.

TABLE II
Counts for the required expression; DFAs are complete over $\{a, b, c\}$.

Representation	$\lvert Q \rvert$	Symbol edges	ε-edges	Total $\lvert \delta \rvert$
Thompson NFA	26	9	20	29
Converted 2-NFA	26	9	20	29
ε-free NFA (= its 2-NFA)	10	14	0	14
Complete subset DFA	11	33	0	33
Minimal complete DFA	7	21	0	21

TABLE III
SUBSET CONSTRUCTION FOR THE REQUIRED EXPRESSION. D_0 IS THE START STATE, D_3 IS THE DEAD STATE, AND $*$ MARKS ACCEPTING STATES (THOSE CONTAINING q_{25}).

DFA state	NFA subset	a	b	c
$\to D_0$	$\{q_0, q_2, q_4\}$	D_1	D_2	D_3
D_1	$\{q_1, q_5, q_6\}$	D_3	D_3	D_4
D_2	$\{q_3, q_5, q_6\}$	D_3	D_3	D_4
D_3 (dead)	$\emptyset$	D_3	D_3	D_3
D_4	$\{q_7, q_8, q_{10}, q_{12}\}$	D_5	D_6	D_3
D_5	$\{q_9, q_{13}, q_{14}, q_{16}, q_{18}\}$	D_7	D_8	D_3
D_6	$\{q_{11}, q_{13}, q_{14}, q_{16}, q_{18}\}$	D_7	D_8	D_3
D_7	$\{q_{15}, q_{19}, q_{20}, q_{22}, q_{24}\}$	D_9	D_{10}	D_3
D_8	$\{q_{17}, q_{19}, q_{20}, q_{22}, q_{24}\}$	D_9	D_{10}	D_3
D_9^*	$\{q_{21}, q_{25}\}$	D_3	D_3	D_3
D_{10}^*	$\{q_{23}, q_{25}\}$	D_3	D_3	D_3

B. Comparison with the DFA

The subset construction has 11 states in the DFA (Table III). D_0 is the ε-closure of the start state of the NFA (q_4) and $D_3 = \emptyset$ is the dead state. Since there are three symbols, each state has 3 outgoing transitions, and $|\delta| = 33$. State minimization results in 7 states and 21 transition edges. The DFA is small here because the expression has no star, so every word has exactly five symbols. However, with a star the picture completely changes: for $(a|b)^*a(a|b)^{r-1}$ ("the r-th symbol from the end is a"), a DFA needs 2^r states (Hopcroft et al., 2007), while the NFA and its 2-NFA need only $O(r)$ states.

C. An ε-free example where conversion adds states

The given expression, defined by $R = (a|b)c(a|b)(a|b)(a|b)$ over $\Sigma = \{a, b, c\}$, needed no changes. Thus, I could not display the algorithm at work. High degrees appear naturally once ε-edges are removed, and spatial processors such as the Automata Processor run automata without ε-moves (Dlugosch et al., 2014).

In order to understand the effect of the algorithm, I first defined a regular expression of $R_1 = (a|b|c|d)x$ with $\Sigma = \{a, b, c, d, x\}$ and then created a program that removes the ε-edges from the Thompson NFA with the standard ε-closure construction, leaving an NFA with just 6 states, 8 edges and zero (0) ε-edges. Its start state has an out-degree of 4 and its accepting state has an in-degree of 4, hence two sides exceed the limit of a 2-NFA. So, each of those states needs $4 - 2 = 2$ new states, so the value $A = 4$. Hence, after conversion, the 2-NFA has 10 states and 12 edges, four of them ε-edges added by the routing tree; every degree is at most 2, and the language is unchanged (Table IV, Figure 3). The Thompson NFA of the same expression is already degree-2 but has 16 states and 18 edges, so removing ε-edges and then converting gives a smaller bounded machine. The same formulas cover the assignment's motivating case: a state with 15 incoming edges needs $15 - 2 = 13$ merge states and adds at most $\lceil \log_2 15 \rceil - 1 = 3$ extra ε-steps.

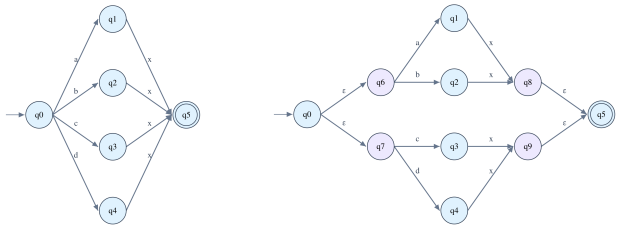


Fig. 3. The ε-free NFA for $(a|b|c|d)x$ before (left) and after (right) conversion. Labels a–d and x are input letters. The 4 purple states are the added splitters and mergers; they are joined by ε-edges, so the 2-NFA is no longer ε-free.

V. CONCLUSION

The ε-splitting and ε-merging of the 2-NFA binary trees transform an NFA into an equivalent 2-NFA with an additional cost of exactly A states and edges/transitions, rather than the exponential explosion into a DFA. The expression does not require any additional space (26 states, or 10 with the removal of ε-edges), while the version without ε-edges adds 66.7% (from 6 to 10 states) and 50% (from 8 to 12 edges) states and edges, respectively, while still remaining below 16 states of a Thompson NFA. The main limitation is that the process takes time, as ε-edges read no input, which means that the signal still take time to cross these edges, so each ε-closure must be solved before the next character. Balanced trees keeps this delay logarithmic, but long ε-chains that were already in the NFA remain.

AI USE DISCLOSURE

I used Claude Code (Opus 5.5) to simplify and clean up my Python code. I also used Claude Opus 5.5 to help rewrite and shorten the text of this paper and to format it in LaTeX. I used Grammarly to improve grammar and wording. I checked every section, ran the code myself to confirm the results, and I take full responsibility for the content.

TABLE IV
THE EXAMPLE $(a|b|c|d)x$: THOMPSON NFA, THE ε-FREE NFA COMPUTED FROM IT, AND THE 2-NFA PRODUCED BY ALGORITHM 1.

Machine	$\lvert Q\rvert$	Symbol edges	ε-edges	$\lvert\delta\rvert$	$\max d^-$	$\max d^+$
Thompson NFA	16	5	13	18	2	2
ε-free NFA	6	8	0	8	4	4
2-NFA from the ε-free NFA	10	8	4	12	2	2

REFERENCES

Brodie, B., Taylor, D., & Cytron, R. (2006). A Scalable Architecture For High-Throughput Regular-Expression Pattern Matching. In *33rd International Symposium on Computer Architecture (ISCA'06)* (p. 191-202). doi: 10.1109/ISCA.2006.7

Dlugosch, P., Brown, D., Glendenning, P., Leventhal, M., & Noyes, H. (2014). An Efficient and Scalable Semiconductor Architecture for Parallel Automata Processing. *IEEE Transactions on Parallel and Distributed Systems*, *25*(12), 3088-3098. doi: 10.1109/TPDS.2014.8

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introduction to Automata Theory, Languages, and Computation* (3rd ed.). Pearson/Addison-Wesley.

Huang, Y., Chen, Z., Li, D., & Yang, K. (2022). CAMA: Energy and Memory Efficient Automata Processing in Content-Addressable Memories. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)* (pp. 25–37). doi: 10.1109/HPCA53966.2022.00011

Kong, L., Yu, Q., Chattopadhyay, A., Le Glaunec, A., Huang, Y., Mamouras, K., & Yang, K. (2022). Software-Hardware Codesign for Efficient In-Memory Regular Pattern Matching. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (pp. 733–748). Association for Computing Machinery. Retrieved from https://doi.org/10.1145/3519939.3523456 doi: 10.1145/3519939.3523456

Kumar, S., Turner, J., & Williams, J. (2006). Advanced algorithms for fast and scalable deep packet inspection. In *Proceedings of the 2006 acm/ieee symposium on architecture for networking and communications systems* (p. 81–92). New York, NY, USA: Association for Computing Machinery. Retrieved from https://doi.org/10.1145/1185347.1185359 doi: 10.1145/1185347.1185359

Le Glaunec, A., Kong, L., & Mamouras, K. (2023, April). Regular Expression Matching using Bit Vector Automata. *Proc. ACM Program. Lang.*, *7*(OOPSLA1). Retrieved from https://doi.org/10.1145/3586044 doi: 10.1145/3586044

Neftci, E., Binas, J., Rutishauser, U., Chicca, E., Indiveri, G., & Douglas, R. J. (2013). Synthesizing cognition in neuromorphic electronic systems. *Proceedings of the National Academy of Sciences*, *110*(37), E3468-E3476. Retrieved from https://www.pnas.org/doi/abs/10.1073/pnas.1212 doi: 10.1073/pnas.1212083110

Schuman, C. D., Potok, T. E., Patton, R. M., Birdwell, J. D., Dean, M. E., Rose, G. S., & Plank, J. S. (2017). *A Survey of Neuromorphic Computing and Neural Networks in Hardware.* Retrieved from https://arxiv.org/abs/1705.06963 (arXiv preprint) doi: 10.48550/arXiv.1705.06963

Thompson, K. (1968). Programming Techniques: Regular Expression Search Algorithm. *Communications of the ACM*, *11*(6), 419–422. Retrieved from https://doi.org/10.1145/363347.363387 doi: 10.1145/363347.363387

Xia, Q., & Yang, J. J. (2019). Memristive Crossbar Arrays for Brain-Inspired Computing. *Nature Materials*, *18*(4), 309–323. Retrieved from https://doi.org/10.1038/s41563-019-0291-x doi: 10.1038/s41563-019-0291-x

Yu, F., Chen, Z., Diao, Y., Lakshman, T. V., & Katz, R. H. (2006). Fast and memory-efficient regular expression matching for deep packet inspection. In *Proceedings of the 2006 ACM/IEEE Symposium on Architecture for Networking and Communications Systems* (pp. 93–102). New York, NY, USA: Association for Computing Machinery. Retrieved from https://doi.org/10.1145/1185347.1185360 doi: 10.1145/1185347.1185360

Zhang, C., Tang, X., & Peng, Y. (2024). Enhancing Regular Expression Processing through Field-Programmable Gate Array-Based Multi-Character Non-Deterministic Finite Automata. *Electronics*, *13*(9). Retrieved from https://www.mdpi.com/2079-9292/13/9/1635 doi: 10.3390/electronics13091635